

Programación

EXAMEN PRÁCTICO – RECUPERACIÓN TEMAS 1-7

(28/05/2026)



LEE ATENTAMENTE LAS SIGUIENTES INSTRUCCIONES ANTES DE EMPEZAR:



- Si estás leyendo esto es porque ya te has descargado el enunciado del examen. Así que, desde este momento **ya puedes desconectar el cable Ethernet de tu equipo.**
- Cuando termines de resolver el examen, **avisa a tu profe** para conectar de nuevo el cable Ethernet y realizar la entrega.
- **Sube el proyecto Java comprimido (.zip) a la entrega de AULES disponible.**

PARTE 1: Configuración del entorno

1. Crea un nuevo proyecto Java (Maven) con IntelliJ -o el IDE que utilices-. Llámalo "INFO MUTXAMEL".
2. Crea en el proyecto un paquete nuevo llamado "comunicados" para ir añadiendo las clases correspondientes al problema planteado.

PARTE 2: Resolución de problemas

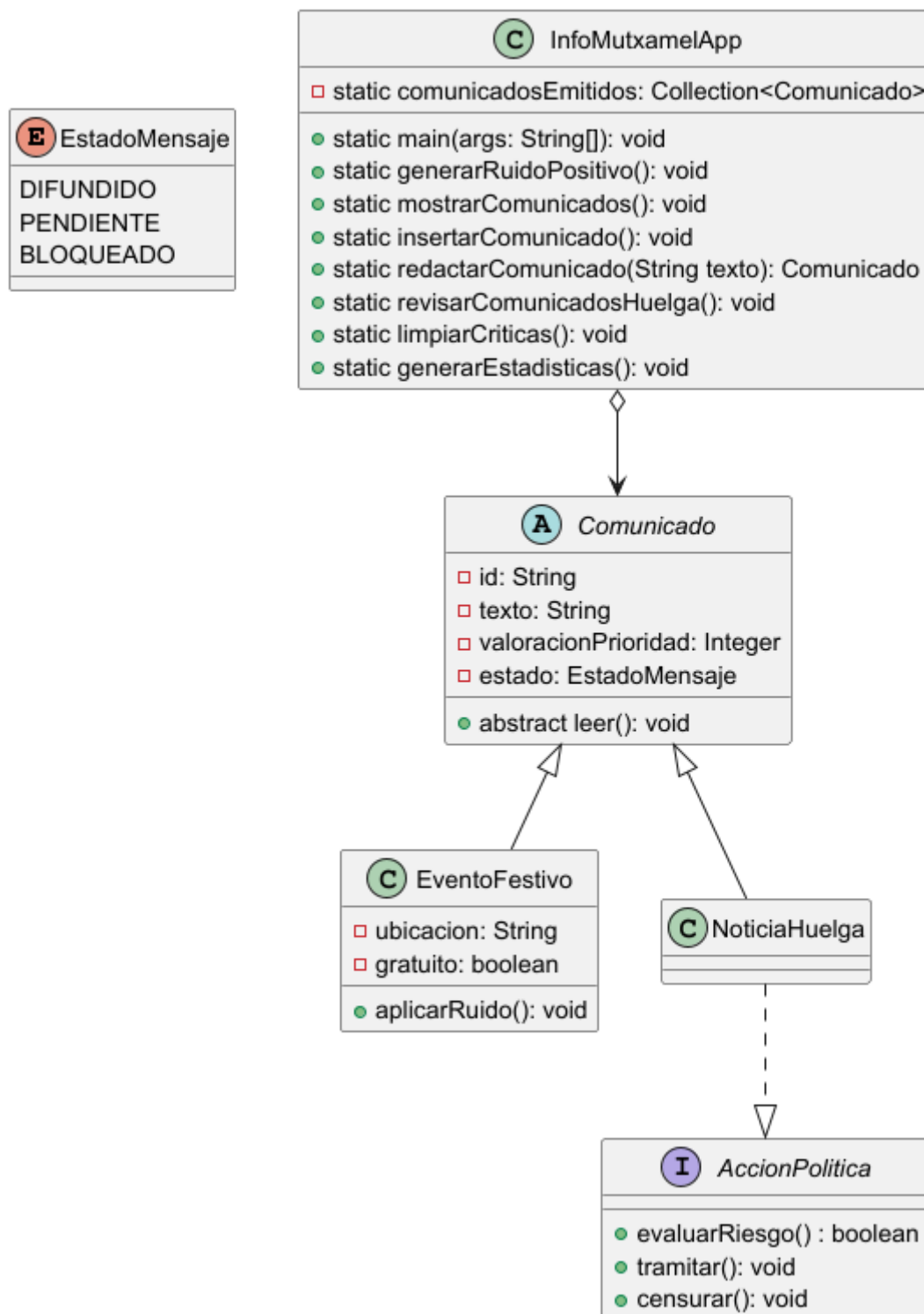
Programa en Java la solución al siguiente problema. Usa el proyecto que te acabas de crear en el apartado anterior.

PROBLEMA: SISTEMA DE CONTROL DE RETÓRICA

El Alcalde de Mutxamel quiere una herramienta para gestionar los mensajes que salen del ayuntamiento. El objetivo es que los mensajes de eventos festivos "tapan" visualmente la información sobre la huelga de profesores en el IES Mutxamel, por lo que necesita un software para priorizar noticias positivas y "gestionar" (filtrar o invisibilizar) el impacto mediático de las protestas docentes.



El diagrama que ha pensado el informático del consistorio para la futura *app* es el siguiente:



Realiza un proyecto en *Java* para implementar la aplicación dada. Tienes libertad para modificar cualquier parte de la estructura, siempre que respetes la funcionalidad que se pide a continuación. Usa *Lombok* si te resulta útil. No se permite el uso de funciones *lambdas* y *streams*.

Condiciones para la construcción de las clases y otras cosas a tener en cuenta (LEE BIEN)

1. (1,5p) Crea la estructura de clases correspondiente al diagrama UML proporcionado.

2. (1,5p) El concejal responsable de comunicación nos pide un método para generar noticias positivas “a cascoporro” y usarlo cuando quiera. Usaremos el método **generarRuidoPositivo()** que se encargará de insertar en la lista de *comunicadosEmitidos* un total de 5 objetos **Comunicado** de tipo **EventoFestivo**.

- El **id** se generará automáticamente con el formato *CM1, CM2, CM3,...* (números correlativos).
- El **texto** será generado aleatoriamente, con 100 caracteres.

Usa esto:

String letras = “ ABCDEFGHI JKLMNO PQRSTU VWX YZ”

- Al atributo **valoracionPrioridad** se le debe asignar valor 1.
- El **estado** será **PENDIENTE**.
- La **ubicación** será siempre “PLAZA DEL AYUNTAMIENTO” y **gratuito = true** por defecto.
- Antes de añadir a la lista los *Comunicados* que se van generando, se les debe aplicar el método **aplicarRuido()** para modificar el estado a **DIFUNDIDO** automáticamente, saltándose la revisión de la noticia por parte del ayuntamiento.

Ten en cuenta que en la lista no se pueden repetir *Comunicados* con el mismo **id** y deben estar almacenados **ordenados por valoracionPrioridad (de menor a mayor)**. En caso de empate en la ordenación, el siguiente atributo a valorar será el **id**.

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Co
[+] Generando ruido positivo a cascoporro...

EventoFestivo{Comunicado{id='CM1', texto='UH      M    NAJ    WF U    SWT    DAN T TQ L HOQ ML BQ H    K CX
EventoFestivo{Comunicado{id='CM2', texto='PV LHSXR IJM P    XDH    X B    JCHCM LVQJC    B GBVM    TO MXO
EventoFestivo{Comunicado{id='CM3', texto='    T    G    MFVQ U    NKK ARGC    FF T    LG ROE JMGQM P P T
EventoFestivo{Comunicado{id='CM4', texto='L VDS    X    T    CBHY    N WK    AD P W    BR X    RTDB    TY    V
EventoFestivo{Comunicado{id='CM5', texto='IE    W    H GHR G EYFMXNIDWRC JMEE    V    S    K OJ    E MKJ    JOA
Comunicados festivos insertados con éxito.
```

3. (1p) El método **mostrarComunicados()** debe recorrer la lista de *comunicadosEmitidos* e invocar al método **leer()** de cada uno. Este método se dedicará a mostrar el texto del *Comunicado* y la info de los atributos de las clases hijas.

```

MOSTRANDO LISTA DE COMUNICADOS:

- Leyendo fiesta...
RF  Y S GWQ JKQD PBT WU  N  EA BYQPAR  O W  IU EF  W  MQUBB  IBEK  D IDX MU  OMT U LJ AN  GS - ubicación: Plaza del ayuntamiento - gratuito: true

- Leyendo fiesta...
K JO  AY DE SSTOD CD  VVEB  R VFMS  ACDAUGA  WH DJ  N IP  HT E  ADJ V  V E  V X CKO K  E ABJLK - ubicación: Plaza del ayuntamiento - gratuito: true

- Leyendo fiesta...
JS DX M6IKPRT  CH JVV Q ISO  R YIQ6 RWGQTBALP W ESXB VS  A  FP K UAB  EV PS  AN I  FJPW TVNHE  WRWP - ubicación: Plaza del ayuntamiento - gratuito: true

- Leyendo fiesta...
W  A HK L RVY  K  O X  A  WHJ ABN A GQF W  Y  K  T  YK EX  FUT J WN C  R YR 6IY X  - ubicación: Plaza del ayuntamiento - gratuito: true

- Leyendo fiesta...
OXDS  HT  NNXGUTVBUY  YI 6P  BN  F FW  T I  C WW0BF JDXK  CQIRJYN RMHOP Q QQN  N  B  P - ubicación: Plaza del ayuntamiento - gratuito: true

Process finished with exit code 0

```

4. (2p) El método *insertarComunicado()* empezará preguntando por pantalla el **texto** del comunicado al usuario, que deberá introducirlo por teclado:

```

INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
Hola quién va a pensar en los niños?

```

A continuación, invocará al método *redactarComunicado(String texto)*, que pedirá al usuario elegir el tipo de *Comunicado* a crear: “**FIESTAS**” ò “**HUELGA**”. Si se introduce un tipo que no es uno de esos dos, el programa deberá insistir hasta que se introduzca un valor válido.

```

INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
Hola quién va a pe
Elige el tipo de comunicado [FIESTAS, HUELGA]:
HOLA
Tipo no válido. Inserta un valor correcto...
Elige el tipo de comunicado [FIESTAS, HUELGA]:

```

Según el tipo de comunicado elegido, se creará:

- Si el tipo es “FIESTAS”, un objeto de tipo **EventoFestivo** con el *texto* recibido, un *id* autogenerado, una *valoracionPrioridad* entre 1 y 4 y *estado* = “DIFUNDIDO”. La *ubicación* y si es *gratuito* se debe preguntar al usuario.

```

INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
hola viva la santa faz
Elige el tipo de comunicado [FIESTAS, HUELGA]:
FIESTAS
Inserta la ubicación:
Alicante
La actividad es gratuita? (S/N):
S
Comunicado insertado. EventoFestivo{Comunicado{id='CM6', texto='hola viva la santa faz', valoracionPrioridad=2, estado=DIFUNDIDO}ubicacion='Alicante', gr

```

- Si el tipo es “HUELGA”, un objeto de tipo **NoticiaHuelga** con el *texto* recibido, un *id* autogenerado, una *valoracionPrioridad* entre 5 y 10 y *estado* = “PENDIENTE”.

El método *redactarComunicado(String texto)* devolverá el objeto de tipo *Comunicado* que se haya creado al método *insertarComunicado()*, y este terminará su trabajo insertando el objeto recibido en la lista de *comunicadosEmitidos*.

```

INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
Hola quién va a pensar en los niños?
Elige el tipo de comunicado [FIESTAS, HUELGA]:
HUELGA
Comunicado insertado. NoticiaHuelga{Comunicado{id='CM6', texto='Hola quién va a pensar en los niños?', valoracionPrioridad=5, estado=PENDIENTE}

```

5. (1,5p) El método *revisarComunicadosHuelga()* recorrerá los *Comunicados* de tipo *NoticiaHuelga* de la lista. **A todos los que tengan el estado = “PENDIENTE”**, deberemos aplicarle la validación de contenido de la siguiente manera:

- Llamaremos al método *evaluarRiesgo()*. Su objetivo es analizar el contenido del texto para detectar posibles daños a la imagen del ayuntamiento, por lo que **debe buscar en el texto las siguientes palabras clave: “ratio”, “manifestación” y “huelga”**. Si se detectan, el método devuelve *true*. Si no, devuelve *false*.
- Si el método anterior devuelve *true*, invocaremos al método *censurar()* para modificar el *estado* del comunicado a “BLOQUEADO”.

```

INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
Hola huelga!
Elige el tipo de comunicado [FIESTAS, HUELGA]:
HUELGA
Comunicado insertado. NoticiaHuelga{Comunicado{id='CM6', texto='Hola huelga!', valoracionPrioridad=7, estado=PENDIENTE}

REVISANDO COMUNICADOS...
Censurando...NoticiaHuelga{Comunicado{id='CM6', texto='Hola huelga!', valoracionPrioridad=7, estado=PENDIENTE}

```

- Si el método anterior devuelve *false*, invocaremos al método **tramitar()** para modificar el *estado* del comunicado a “**DIFUNDIDO**”.

```
INSERTANDO NUEVO COMUNICADO...
Escribe el texto:
hola quién va a pensar en los niños
Elige el tipo de comunicado [FIESTAS, HUELGA]:
HUELGA
Comunicado insertado. NoticiaHuelga{Comunicado{id='CM6', texto='hola quién va a pensar en los niños', valoracionPrioridad=9, estado=PENDIENTE}}

REVISANDO COMUNICADOS...
Tramitando...NoticiaHuelga{Comunicado{id='CM6', texto='hola quién va a pensar en los niños', valoracionPrioridad=9, estado=PENDIENTE}}
```

6. (1p) El método **limpiarCriticas()** debe recorrer la lista de comunicados para borrar todos aquellos cuyo *estado* sea “**BLOQUEADO**”.
7. (1,5p) Por último, implementa el método **generarEstadisticas()**. Este método debe:
 - Crear una matriz de tipo *double* y tamaño 3x2, donde las filas son los tres estados del *enum* (DIFUNDIDO, PENDIENTE, BLOQUEADO).
 - Recorrer la lista para contabilizar en la columna 0 cuántos comunicados hay en cada *estado* y rellenar la matriz.
 - Calcular en la columna 1 el porcentaje que representa cada *estado* respecto al total de comunicados.

Por ejemplo, si en el sistema hubiera 10 comunicados registrados (2 están **BLOQUEADOS**, 3 están **PENDIENTES** y 5 está **DIFUNDIDO**), la matriz en memoria se gestionaría de la siguiente forma:

Índice (Fila)	Col 0 (Conteo)	Col 1 (Porcentaje)
Fila 0 (DIFUNDIDO)	5.0	50.0%
Fila 1 (PENDIENTE)	3.0	30.0%
Fila 2 (BLOQUEADO)	2.0	20.0%

- Muestra por pantalla un resumen de las estadísticas calculadas:

```
Estadísticas de comunicados:
- DIFUNDIDO: 5 mensajes (50.00%)
- PENDIENTE: 3 mensajes (30.00%)
- BLOQUEADO: 2 mensajes (20.00%)
-----
Total de comunicados procesados: 10
```